

UniPeek — User Manual

Stream your Unity Game View live to your iOS or Android phone over Wi-Fi.

What You Need

- A phone with the **UniPeek** companion app installed (iOS or Android)
 - Both your PC/Mac and phone on the **same Wi-Fi network**
 - Unity 2021.3 or newer
-

Opening the UniPeek Window

Go to **Window** → **UniPeek** in the Unity menu bar.

The UniPeek window can be docked anywhere in your editor layout like any other Unity panel.

Connecting Your Phone

Option 1 — QR Code (easiest)

1. Click **Start Streaming** in the UniPeek window.
2. A QR code appears in the window.
3. Open the UniPeek app on your phone and tap **Scan QR**.
4. Point the camera at the QR code.
5. The dot in the window turns green — you are connected.

Option 2 — Auto-Discovery (mDNS)

UniPeek broadcasts its presence on the local network automatically. In the UniPeek app, tap **Browse** and your machine name will appear in the list. Tap it to connect.

No QR code needed. Both devices must be on the same subnet.

Option 3 — Reverse Connection (Android) — Under development

This feature is currently under development and may not work in all builds.

Use this when your phone cannot reach your PC (hotel Wi-Fi, corporate network with client isolation).

1. In the UniPeek app, switch to **Listen** mode.
2. In Unity, click **Start Streaming**, then expand the **Reverse Connection** section.
3. Enter your phone's IP address and click **Connect to Phone**.
4. The editor dials out to the phone on port **7778**.

Option 4 — USB / ADB (Android only) — Coming soon

Automatic ADB port forwarding is not yet built into the plugin. As a workaround, run the following command manually after connecting via USB, then connect the app to **localhost**:

```
adb reverse tcp:7777 tcp:7777
```

The Editor Window at a Glance

Area	What it does
Status dot	Grey = stopped, Amber = waiting for phone, Green = connected
Start / Stop Streaming button	Starts or stops the stream
QR code	Appears while waiting; disappears once connected
Stats bar	Shows live FPS and encode time (WebRTC mode shows RTT instead)
Options section	Editor name, socket mode, play mode behaviour, capture method, log level
Connected Devices list	Shows all currently connected phones
Reverse Connection panel	Manual outbound connection to a phone in Listen mode
Docs button	Opens this manual online
Reset FW button	Re-runs Windows Firewall setup if connections are failing
Port field	Configurable listen port (default 7777); editable only when not streaming

Options

Editor Name

Sets the display name shown in the UniPeek app's device list. Defaults to your machine name. Type a name and click **Set** to save it.

Socket Mode

Selects the streaming transport:

Mode	Description
WebSocket	JPEG frames sent as binary WebSocket messages. Works on all setups.
WebRTC	Low-latency peer-to-peer video. Requires com.unity.webrtc ≥ 3.0.0 in your project.

The WebRTC option only appears if the **com.unity.webrtc** package is installed.

WebRTC mode runs in Play Mode and captures the composited Game View at end-of-frame, preserving Screen Space Overlay canvases and UniPeek touch gizmos. The optional **STUN URL** setting can help on VPNs, hotspots, or unusual subnet setups; leave it empty for local-only LAN behavior.

Run in Play Mode

When **on**: streaming only runs while the Editor is in Play Mode. Entering Edit Mode automatically stops streaming.

When **off**: streaming runs all the time, even in Edit Mode. The stream will briefly drop when scripts recompile (domain reload). Useful for inspecting the editor camera without entering Play.

Capture Method

Method	Description
Camera Render	Renders the main camera synchronously. Works in Edit and Play Mode.
Async GPU Readback	Same render path but non-blocking — reduces CPU stall at the cost of ~1 frame of extra latency.

Switch between them at any time; the change takes effect on the next captured frame.

Log Level

Controls how much UniPeek writes to the Unity Console:

Level	Output
None	Completely silent
Error	Errors only
Warning	Errors and warnings
All	Full diagnostic output

Touch Input

UniPeek injects touch into **every active input backend** simultaneously:

Backend	How it works	Reliability
New Input System (<code>com.unity.inputsystem</code>)	Virtual <code>Touchscreen</code> device via <code>InputSystem.QueueStateEvent</code>	Fully supported
Legacy Input Manager	Internal <code>Input.SimulateTouch(Touch)</code> via reflection	Best-effort — works on Unity 2021 – 2022 – 6; may break on future versions

The active backend is controlled by **Edit** → **Project Settings** → **Player** → **Active Input Handling**:

- **Input System Package (New)** — only the new Input System path runs.
- **Input Manager (Old)** — only the Legacy reflection path runs.
- **Both** — UniPeek injects into both systems at the same time. This is the recommended setting when your project uses a mix of old and new Input APIs.

UniPeek delivers touches through two additional channels regardless of backend:

Channel	Class / API
UniPeekInput events	<code>UniPeekInput.OnTouch</code> / <code>OnTouchDetailed</code>
Unity Input System polling	<code>ETouch.activeTouches</code> , <code>Touchscreen.current</code> , etc. (requires new Input System)

Touch overlays (semi-transparent circles) are drawn on the Game View automatically while touches are active.

Multi-touch, gyroscope, and accelerometer are **Pro** features. Free tier receives single touch only.

UniPeekInput

`UniPeekInput` is a lightweight static event bus. UniPeek fires its events on the main thread for every touch that arrives from the phone.

```
using UniPeek;
using UnityEngine;

public class Example : MonoBehaviour
{
    void OnEnable() => UniPeekInput.OnTouch += HandleTouch;
    void OnDisable() => UniPeekInput.OnTouch -= HandleTouch;

    void HandleTouch(Vector2 normalizedPos)
    {
        // normalizedPos.x : 0 = left edge, 1 = right edge
        // normalizedPos.y : 0 = top edge, 1 = bottom edge
        float screenX = normalizedPos.x * Screen.width;
        float screenY = (1f - normalizedPos.y) * Screen.height; // flip Y for
        // Unity screen space
        Debug.Log($"Touch at screen ({screenX}, {screenY})");
    }
}
```

OnTouchDetailed

Use this when you need the finger ID or phase string:

```
UniPeekInput.OnTouchDetailed += (fingerId, phase, normalizedPos) =>
{
    // phase: "began" | "moved" | "ended" | "canceled"
```

```
Debug.Log($"Finger {fingerId} {phase} at {normalizedPos}");  
};
```

When to use UniPeekInput

- You want a simple callback without dealing with the Input System device layer.
- You need to react to individual touch events (e.g. "tap began") rather than polling state every frame.

New Input System integration

When `com.unity.inputsystem` is installed, UniPeek automatically injects all touches into a virtual `Touchscreen` device. You can read them using any standard Input System API.

Single touch — polling

```
using UnityEngine.InputSystem;  
  
var ts = Touchscreen.current;  
if (ts != null && ts.primaryTouch.press.isPressed)  
{  
    Vector2 pos = ts.primaryTouch.position.ReadValue();  
}
```

Multi-touch — Enhanced Touch

Enable `EnhancedTouchSupport` once (e.g. in `OnEnable`) then read `ETouch.activeTouches` every frame:

```
using UnityEngine.InputSystem.EnhancedTouch;  
using ETouch = UnityEngine.InputSystem.EnhancedTouch.Touch;  
  
void OnEnable() => EnhancedTouchSupport.Enable();  
void OnDisable() => EnhancedTouchSupport.Disable();  
  
void Update()  
{  
    var touches = ETouch.activeTouches;  
  
    // Filter out Ended/Canceled – they linger for one frame after lift  
    int live = 0;  
    foreach (var t in touches)  
    {  
        if (t.phase == UnityEngine.InputSystem.TouchPhase.Ended ||  
            t.phase == UnityEngine.InputSystem.TouchPhase.Canceled) continue;  
        live++;  
    }  
}
```

Important: Always filter out **Ended** and **Canceled** phases when counting active fingers. Enhanced Touch keeps a lifted touch in **activeTouches** for one frame after it ends, which can cause spurious multi-touch events if you only check **Count**.

Coordinate system

UniPeek's phone sends **normalised** coordinates where **(0, 0)** is the **top-left** of the screen. Unity's Input System uses **screen pixels** where **(0, 0)** is the **bottom-left**.

UniPeek applies the conversion automatically before injecting into the Input System:

```
screenX = normalizedX × Screen.width
screenY = (1 - normalizedY) × Screen.height    // Y-flip
```

When reading via **UniPeekInput**, the coordinates are still in the raw normalised form (Y = 0 at top). Apply the same flip yourself when converting to screen or world space.

Pinch-to-scale example

The finger-pair baseline must be reset whenever the tracked pair changes (e.g. a third finger joins, or one of the original two lifts while the other stays down).

```
private float _prevPinchDist = -1f;
private int _pinchId0 = -1, _pinchId1 = -1;

void Update()
{
    var touches = ETouch.activeTouches;

    ETouch t0 = default, t1 = default;
    int live = 0;
    for (int i = 0; i < touches.Count && live < 2; i++)
    {
        var ph = touches[i].phase;
        if (ph == UnityEngine.InputSystem.TouchPhase.Ended ||
            ph == UnityEngine.InputSystem.TouchPhase.Canceled) continue;
        if (live == 0) t0 = touches[i]; else t1 = touches[i];
        live++;
    }

    if (live >= 2)
    {
        // Reset when the finger pair changes
        if (t0.touchId != _pinchId0 || t1.touchId != _pinchId1)
        {
            _prevPinchDist = -1f;
            _pinchId0 = t0.touchId;
            _pinchId1 = t1.touchId;
        }
    }
}
```

```

float dist = Vector2.Distance(t0.screenPosition, t1.screenPosition);
if (_prevPinchDist > 0f)
{
    float ratio = dist / _prevPinchDist;
    float s = Mathf.Clamp(transform.localScale.x * ratio, 0.1f, 5f);
    transform.localScale = Vector3.one * s;
}
_prevPinchDist = dist;
}
else
{
    _prevPinchDist = -1f;
    _pinchId0 = _pinchId1 = -1;
}
}

```

Touch Input quick reference

What you want	Recommended API	Tier
Simple tap callback	<code>UniPeekInput.OnTouch</code>	Free + Pro
Phase + finger ID	<code>UniPeekInput.OnTouchDetailed</code>	Free + Pro
Single-touch polling	<code>Touchscreen.current.primaryTouch</code>	Free + Pro
Multi-touch / pinch	<code>ETouch.activeTouches</code> (Enhanced Touch)	Pro

Gyroscope and Accelerometer

The phone continuously sends gyroscope (rotation rate) and accelerometer (gravity + motion) data to Unity. These are **Pro** features. When `com.unity.inputsystem` is installed, they are injected as virtual sensor devices — your game code reads them as normal.

Accelerometer

```

// New Input System (required)
var accel = Accelerometer.current;
if (accel != null)
    Vector3 g = accel.acceleration.ReadValue();

```

Gyroscope

```

// New Input System — rotation rate via AttitudeSensor
var attitude = AttitudeSensor.current;

```

```
if (attitude != null)
    Quaternion rot = attitude.attitude.ReadValue();
```

Legacy `Input.gyro` injection is not supported by Unity's public API. Use the new Input System for gyroscope data.

Windows Firewall

On first launch, UniPeek asks for a one-time UAC (administrator) prompt to add a Windows Firewall rule allowing inbound connections on port **7777**. Without this, the phone cannot reach the editor.

If you declined the prompt or the rule was removed, click **Reset FW** in the toolbar and then **Start Streaming** again to re-run the setup.

If you prefer to add the rule manually, run this in an elevated PowerShell:

```
New-NetFirewallRule -DisplayName "UniPeek" -Direction Inbound -Action Allow -
Protocol TCP -LocalPort 7777 -Profile Any
```

Troubleshooting

Symptom	Fix
QR code shows <code>127.0.0.1</code>	Your machine has no active Wi-Fi or Ethernet. Connect to the network first.
Phone can't find the host via Browse	Both must be on the same subnet. Some guest/corporate Wi-Fi blocks device-to-device traffic — try the QR code or Reverse Connection mode instead.
Firewall prompt never appeared / connections fail	Click Reset FW in the toolbar, then Start Streaming again.
Stream is black or frozen	Make sure there is at least one camera tagged <code>MainCamera</code> in your scene. In Edit Mode, <code>Camera.main</code> must exist.
Touch events are not registering	Check Edit → Project Settings → Player → Active Input Handling . If using Legacy only, UniPeek injects via reflection (best-effort). For reliable injection, install <code>com.unity.inputsystem</code> and set Active Input Handling to Input System Package or Both .
Touch works but UI / Canvas buttons don't respond	Your Canvas is still using the old Standalone Input Module . Select the EventSystem in your scene and replace the Standalone Input Module component with Input System UI Input Module (<code>UnityEngine.InputSystem.UI.InputSystemUIInputModule</code>).

Symptom	Fix
High latency or choppy video	Lower the quality setting in the app, switch to Async GPU Readback , or reduce resolution to 540p.
Stream drops on recompile	Disable Run in Play Mode to allow the stream to persist across domain reloads.
WebRTC mode not available	Install <code>com.unity.webrtc</code> $\geq 3.0.0$. The Socket Mode dropdown will show the WebRTC option once the package is detected.

Default Ports

Port	Purpose
7777	UniPeek WebSocket server (phone → editor, configurable)
7778	Reverse connection (editor → phone in Listen mode, fixed)